Звіт з лабораторної роботи №2

Дорошенко 301-ТК

Багатопоковість. Асинхроність. IEnumerables. LINQ.

Робота буде проходити у тому ж рішенні University, що і минула лабораторна робота.

Створюю асинхронний інтерфейс для файлу ICrudServiceAsync.cs та CrudServiceAsync.cs:

University.Common

```
using System;
using System.Collections;
using System.Collections.Generic;
using System.Linq;
using System.Text.Json;
using System.Threading.Tasks;
using University.Common.Models;

namespace University.Common.Services
{
    1 reference
    public class CrudServiceAsync<T> : ICrudServiceAsync<T>
    {
        private readonly List<T> _items = new();
        private readonly object _lock = new object(); // для thread-safety
        private readonly string _filePath;

        0 references
        public CrudServiceAsync(string filePath = null)
        {
            _filePath = filePath ?? $"data_{typeof(T).Name}.json";
        }

        1 reference
        public async Task<bool> CreateAsync(T element)
        {
            if (element == null) return false;

            await Task.Run(() =>
            {
                lock (_lock)
                {
                    _items.Add(element);
                }
            });

            Console.WriteLine($"[+] Асинхронно створено {typeof(T).Name}");
            return true;
        }

        1 reference
        public async Task<T?> ReadAsync(Guid id)
        {
            return await Task.Run(() =>
            {
                lock (_lock)
                {
                    var idProperty = typeof(T).GetProperty("Id");
                    return _items.FirstOrDefault(item =>
                    {
                        var itemId = (Guid?)idProperty?.GetValue(item);
                        return itemId == id;
                    });
                }
            });
        }

        1 reference
        public async Task<IEnumerable<T>> ReadAllAsync()
        {
            return await Task.Run(() =>
            {
                lock (_lock)
                {
                    return _items.ToList();
                }
            });
        }

        1 reference
        public async Task<IEnumerable<T>> ReadAllAsync(int page, int amount)
        {
            return await Task.Run(() =>
            {
                lock (_lock)
                {
                    return _items.Skip(page * amount).Take(amount).ToList();
                }
            });
        }
```

```
69                  lock (_lock)
70                  {
71                      return _items.Skip(page * amount).Take(amount).ToList();
72                  }
73              });
74          }
75
            1 reference
76          public async Task<bool> UpdateAsync(T element)
77          {
78              if (element == null) return false;
79
80              return await Task.Run(() =>
81              {
82                  lock (_lock)
83                  {
84                      var idProperty = typeof(T).GetProperty("Id");
85                      if (idProperty == null) return false;
86
87                      var id = (Guid)idProperty.GetValue(element)!;
88                      var index = _items.FindIndex(item => (Guid)idProperty.GetValue(item)! == id);
89
90                      if (index != -1)
91                      {
92                          _items[index] = element;
93                          return true;
94                      }
95                      return false;
96                  }
97              });
98          }
99
            1 reference
100         public async Task<bool> RemoveAsync(T element)
101         {
102             if (element == null) return false;
103
104             return await Task.Run(() =>
105             {
106                 lock (_lock)
107                 {
108                     return _items.Remove(element);
109                 }
110             });
111         }
112
            1 reference
113         public async Task<bool> SaveAsync()
114         {
115             return await Task.Run(() =>
116             {
117                 try
118                 {
119                     lock (_lock)
120                     {
121                         string json = JsonSerializer.Serialize(_items, new JsonSerializerOptions
122                         {
123                             WriteIndented = true
124                         });
125                         File.WriteAllText(_filePath, json);
126                         Console.WriteLine($"[✅] Дані збережено у файл: {_filePath}");
127                         return true;
128                     }
129                 }
130                 catch (Exception ex)
131                 {
132                     Console.WriteLine($"Помилка збереження: {ex.Message}");
133                     return false;
134                 }
135             });
136         }
137
138         // Реалізація IEnumerable<T>
            1 reference
139         public IEnumerator<T> GetEnumerator() => _items.GetEnumerator();
            0 references
140         IEnumerator IEnumerable.GetEnumerator() => GetEnumerator();
141     }
142 }
```

```
CrudServiceAsync.cs   ICrudServiceAsync.cs   CrudService.cs   ICrudService.cs   Course.cs   Teacher.cs   Student.cs   En
University.Common                                University.Common.Services.ICrudServiceAsync<T>
1   using System;
2   using System.Collections.Generic;
3   using System.Threading.Tasks;
4
5   namespace University.Common.Services
6   {
        0 references
7       public interface ICrudServiceAsync<T> : IEnumerable<T>
8       {
            0 references
9           Task<bool> CreateAsync(T element);
            0 references
10          Task<T?> ReadAsync(Guid id);
            0 references
11          Task<IEnumerable<T>> ReadAllAsync();
            0 references
12          Task<IEnumerable<T>> ReadAllAsync(int page, int amount);
            0 references
13          Task<bool> UpdateAsync(T element);
            0 references
14          Task<bool> RemoveAsync(T element);
            0 references
15          Task<bool> SaveAsync();          // збереження у файл
16      }
17  }
```

Додаю статичні методи на прикладі оновлення файлу Student.cs:

```
27          public static Student CreateNew()
28          {
29              var random = new Random();
30              string[] firstNames = { "Петро", "Анна", "Іван", "Марія", "Олег", "Софія", "Дмитро", "Катерина" };
31              string[] lastNames = { "Мельник", "Бондар", "Шевченко", "Коваленко", "Ткачук", "Кравченко", "Сидоренко" };
32
33              return new Student
34              {
35                  FirstName = firstNames[random.Next(firstNames.Length)],
36                  LastName = lastNames[random.Next(lastNames.Length)],
37                  Course = random.Next(1, 7),
38                  GPA = Math.Round(3.0 + random.NextDouble() * 2.0, 2), // від 3.0 до 5.0
39                  StudentId = $"STU-{DateTime.Now.Year}-{random.Next(10000, 99999)}"
40              };
41          }
42      }
43  }
```

Реалізація програми та паралельне створення 1000+ студентів (лістинг знизу):

```
=== Лабораторна робота №2: Асинхронний CRUD + Parallel ===

Паралельне створення 1000 студентів...

=== Клас Person був ініціалізований ===
[+] Асинхронно створено Student
[+] Асинхронно створено Student
[+] Асинхронно створено Student
```

```
using System;
using System.Diagnostics;
using System.Linq;
using System.Threading.Tasks;
using University.Common.Models;
using University.Common.Services;
using University.Common.Extensions;

namespace University.ConsoleApp
{
  internal class Program
   {
      static async Task Main(string[] args)
      {
         Console.OutputEncoding = System.Text.Encoding.UTF8;
         Console.InputEncoding = System.Text.Encoding.UTF8;

         Console.WriteLine("=== Лабораторна робота №2: Асинхронний CRUD +
Parallel ===\n");

         // Створюємо асинхронний сервіс
         var studentService = new CrudServiceAsync<Student>("students.json");

         var stopwatch = Stopwatch.StartNew();

         Console.WriteLine("Паралельне створення 1000 студентів...\n");

         // Паралельне створення 1000 студентів
         await Parallel.ForEachAsync(
            Enumerable.Range(1, 1000),
            new ParallelOptions { MaxDegreeOfParallelism = 20 }, // обмежуємо
кількість потоків
            async (i, token) =>
            {
               var student = Student.CreateNew();
               await studentService.CreateAsync(student);
            });
```

```
        stopwatch.Stop();

        // Результати
        var allStudents = (await studentService.ReadAllAsync()).ToList();

        Console.WriteLine($"Час виконання: {stopwatch.ElapsedMilliseconds} мс");
        Console.WriteLine($"Успішно створено студентів: {allStudents.Count}");
        Console.WriteLine($"Мінімальний GPA: {allStudents.Min(s => s.GPA)}");
        Console.WriteLine($"Максимальний GPA: {allStudents.Max(s => s.GPA)}");
        Console.WriteLine($"Середній GPA: {allStudents.Average(s => s.GPA):F2}");

        // Збереження у файл
        await studentService.SaveAsync();

        Console.WriteLine("\n=== Лабораторна робота №2 виконана успішно! ===");
        Console.ReadKey();
    }
  }
}
```